Red Hat Lightspeed cost management API

Pau Garcia Quiles, Senior Principal Product Manager, Red Hat Lightspeed

Table of Contents

Introduction

The Cost Management API allows users to obtain all the data available in the UI, and even more since with the API it's possible to combine filters or to make several API calls and then combine the results.

The latest version of this cheat sheet, and its accompanying Bruno collection, are always available from `https://github.com/project-koku/costmgmt-api-cheatsheet`.

Below are common scenarios with example API calls.

Base URL: `https://console.redhat.com`

Authentication

Using service account tokens (OAuth2 client credentials)

Grab your token using the following curl command:

```
curl -v \
  -d "client_id=CLIENT_ID" \
  -d "client_secret=CLIENT_SECRET" \
  -d "grant_type=client_credentials" \
  -d "scope=api.console" \
  "https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-connect/token"
```

Use the token in an API request via curl:

```
curl --location \
  'https://console.redhat.com/api/cost-management/v1/reports/aws/costs/' \
  --header 'Authorization: Bearer ACCESS_TOKEN'
```

OpenShift Costs

Cluster costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=*
```

Cluster costs only for the current month:

```
GET /api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&group_by[cluster]=*
```

Project costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[project]=*
```

Project costs grouped by cluster

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=*&group_by[project]=*
```

Project costs grouped by tag

Costs by project, grouped by a tag key (e.g. `application`):

```
GET /api/cost-
management/v1/reports/openshift/costs/?group_by[project]=*&group_by[tag:application]=*
```

Node costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[node]=*
```

Tag costs

Costs grouped by a tag key (e.g. `env`):

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[tag:env]=*
```

OpenShift Virtualization VMs

```
GET /api/cost-management/v1/reports/openshift/resources/virtual-machines/
```

Optionally filter by cluster:

```
GET /api/cost-management/v1/reports/openshift/resources/virtual-
machines/?filter[exact:cluster]=CLUSTER_UUID
```

GPU costs

```
GET /api/cost-management/v1/reports/openshift/gpu/
```

GPU costs by MIG profile:

```
GET /api/cost-management/v1/reports/openshift/gpu/mig_profiles/
```

CPU

```
GET /api/cost-management/v1/reports/openshift/compute/
```

Memory

```
GET /api/cost-management/v1/reports/openshift/memory/
```

Storage volumes

```
GET /api/cost-management/v1/reports/openshift/volumes/?group_by[storageclass]=*
```

Storage volumes for a specific project (e.g. `cost-management`):

```
GET /api/cost-management/v1/reports/openshift/volumes/?filter[exact:project]=cost-
management&group_by[storageclass]=*
```

Breakdown page: costs for a cluster

Cost breakdown for the "demolab" cluster:

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=demolab
```

Breakdown page: costs for a project

Cost breakdown for the "cost-management" project:

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[project]=cost-management
```

Cloud Filtered by OpenShift

The following endpoints return cloud data specifically relating to OpenShift clusters. They allow for both cluster and cloud group_by/filtering options (cluster, node, project, account, service, region).

```
GET /api/cost-management/v1/reports/openshift/infrastructures/all/costs/
GET /api/cost-management/v1/reports/openshift/infrastructures/aws/costs/
GET /api/cost-management/v1/reports/openshift/infrastructures/azure/costs/
GET /api/cost-management/v1/reports/openshift/infrastructures/gcp/costs/
```

AWS Costs

Per AWS account

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[account]=*
```

Per AWS region

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[region]=*
```

Per AWS service

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[service]=*
```

Per AWS EC2 VM

```
GET /api/cost-management/v1/reports/aws/resources/ec2-compute/
```

Filtering by operating system (e.g. Fedora):

```
GET /api/cost-management/v1/reports/aws/resources/ec2-compute/?filter[operating_system]=Fedora
```

Per AWS tag

Costs grouped by an AWS tag key (e.g. `AppTeam`):

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[tag:AppTeam]=*
```

Per AWS cost category

Costs grouped by an AWS cost category (e.g. `CostCenter`):

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[aws_category:CostCenter]=*
```

Per AWS Organizational Unit

Group by root organizational unit "r-f22a":

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[org_unit_id]=r-f22a
```

Per AWS instance type

```
GET /api/cost-management/v1/reports/aws/instance-types/
```

Per AWS storage

```
GET /api/cost-management/v1/reports/aws/storage/
```

AWS cost types

Getting unblended, blended and amortized costs:

```
GET /api/cost-management/v1/reports/aws/costs/?cost_type=unblended_cost
GET /api/cost-management/v1/reports/aws/costs/?cost_type=blended_cost
GET /api/cost-management/v1/reports/aws/costs/?cost_type=savingsplan_effective_cost
```

Azure Costs

Per Azure account (subscription)

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[subscription_guid]=*
```

Per Azure region

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[resource_location]=*
```

Per Azure service

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[service_name]=*
```

Per Azure tag

Costs grouped by an Azure tag key (e.g. `ms-resource-usage`):

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[tag:ms-resource-usage]=*
```

Per Azure instance type

```
GET /api/cost-management/v1/reports/azure/instance-types/
```

Per Azure storage

```
GET /api/cost-management/v1/reports/azure/storage/
```

Google Cloud Costs

Per Google Cloud account

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[account]=*
```

Per Google Cloud region

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[region]=*
```

Per Google Cloud service

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[service]=*
```

Per Google Cloud Project

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[gcp_project]=*
```

Per Google Cloud tag

Costs grouped by a Google Cloud label (e.g. `env`):

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[tag:env]=*
```

Per Google Cloud instance type

```
GET /api/cost-management/v1/reports/gcp/instance-types/
```

Per Google Cloud storage

```
GET /api/cost-management/v1/reports/gcp/storage/
```

Tags

List tag keys and values

Cloud source tags:

```
GET /api/cost-management/v1/tags/aws/
GET /api/cost-management/v1/tags/azure/
GET /api/cost-management/v1/tags/gcp/
```

OpenShift tags (labels):

```
GET /api/cost-management/v1/tags/openshift/
```

You can also do OCP-on-cloud variations by adding `infrastructures/{cloud}` (i.e. `gcp`, `azure`, `aws`).

Additional filtering options

Show disabled tags:

```
GET /api/cost-management/v1/tags/openshift/?filter[enabled]=False
```

Show tags for a particular time frame:

```
GET /api/cost-management/v1/tags/openshift/?start_date=2025-11-15&end_date=2025-12-17
```

See tag values directly for a specific key by appending the key ("my-tag-key") to the URL:

```
GET /api/cost-management/v1/tags/openshift/my-tag-key/
```

Optimizations (Resource Optimization Service)

Container recommendations

All container-level CPU/memory right-sizing recommendations:

```
GET /api/cost-management/v1/recommendations/openshift/
```

Recommendations for a specific container (use the recommendation UUID):

```
GET /api/cost-management/v1/recommendations/openshift/{recommendation_uuid}
```

The detail response includes per-term (short/medium/long) recommendations for two models:

- **Cost model**: optimizes for resource savings (p60 CPU / p95 memory)

- **Performance model**: optimizes for headroom (p98 CPU / p100 memory)

Each recommendation includes CPU/memory requests and limits, notifications, and usage distribution box plots (min, Q1, median, Q3, max).

Filtering and sorting

Filter by cluster, project, workload, or container:

```
GET /api/cost-management/v1/recommendations/openshift/?cluster_uuid=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/?project=my-namespace
GET /api/cost-management/v1/recommendations/openshift/?workload=my-deployment
GET /api/cost-management/v1/recommendations/openshift/?container=my-container
```

Filter by GPU:

```
GET /api/cost-management/v1/recommendations/openshift/?has_gpu=true
GET /api/cost-management/v1/recommendations/openshift/?gpu_model=A100
GET /api/cost-management/v1/recommendations/openshift/?gpu_classification=underutilized
```

Sort by last reported or total savings:

```
GET /api/cost-management/v1/recommendations/openshift/?order_by=last_reported&order_how=desc
```

Replica counts and total savings

Container detail responses include replica information collected from `kube-state-metrics` (deployments, statefulsets, daemonsets):

- `replica_info.desired` – spec-level replica count
- `replica_info.available` – currently available/ready replicas
- `replica_info.source` – `"kube_state_metrics"` (authoritative) or `"derived"` (pod count fallback)

Total impact savings multiply per-container savings by replica count.

Namespace recommendations

Namespace-level CPU/memory recommendations (aggregated across all containers in a namespace):

```
GET /api/cost-management/v1/openshift/namespace/recommendations
```

Detail for a specific namespace recommendation:

```
GET /api/cost-management/v1/recommendations/openshift/namespace/{recommendation_uuid}
```

NVIDIA MIG (Multi-Instance GPU) recommendations

MIG bin-packing recommendations are part of the container detail response. When a container uses a MIG-capable GPU (A100, H100, H200, B200), the response includes a `gpu` object with:

- `current_gpu_model` / `current_gpu_profile` – what the container currently uses
- `recommended_gpu_profile` – smallest MIG profile whose resources cover p95 usage
- `gpu_classification` – e.g. `compute_bound`, `memory_bound`, `underutilized`
- `estimated_monthly_gpu_savings_usd` – dollar savings from MIG right-sizing

```
GET /api/cost-management/v1/recommendations/openshift/{recommendation_uuid}
```

NVIDIA GPU time-slicing recommendations

Node-level GPU time-slicing consolidation guidance. Evaluates how many containers could share each physical GPU via `nvidia.com/gpu.replicas`:

```
GET /api/cost-management/v1/recommendations/openshift/nodes
```

Returns per-node recommendations with:

- `gpu_model` – NVIDIA GPU model on the node
- `recommended_replicas` – suggested time-slicing replica count
- `savings_per_gpu_usd` / `total_node_savings_usd` – estimated cost savings

Node CPU/memory utilization recommendations

Node-level right-sizing based on CPU and memory utilization patterns – identifies underutilized, overcommitted, and stranded resource nodes:

```
GET /api/cost-management/v1/recommendations/openshift/nodes/utilization
```

Filter by underutilized or overcommitted nodes:

```
GET /api/cost-management/v1/recommendations/openshift/nodes/utilization?is_underutilized=true
GET /api/cost-management/v1/recommendations/openshift/nodes/utilization?is_overcommitted=true
```

Returns per-node metrics including CPU/memory utilization percentiles (p50, p95), overcommit ratio, trend slope, pod count, and stranded resource detection.

PVC (Persistent Volume Claim) recommendations

Right-sizing recommendations for persistent volumes – oversized, near-full, orphaned, and growth trend projection:

```
GET /api/cost-management/v1/recommendations/openshift/pvcs
```

Snapshot staleness recommendations

VolumeSnapshot analysis — orphaned, stale, never-restored, redundant snapshots with estimated storage savings:

```
GET /api/cost-management/v1/recommendations/openshift/snapshots
```

Snapshot staleness settings (configurable thresholds):

```
GET /api/cost-management/v1/recommendations/openshift/settings/snapshot
PUT /api/cost-management/v1/recommendations/openshift/settings/snapshot
```

Fleet summary

Cross-cluster aggregated savings, adoption rates, and top optimization opportunities:

```
GET /api/cost-management/v1/recommendations/openshift/fleet-summary
```

Recommendation term settings

Customers can configure their own 3 recommendation term windows (1–90 days each), replacing the fixed short (1d) / medium (7d) / long (15d) defaults:

```
GET /api/cost-management/v1/recommendations/openshift/settings/terms
PUT /api/cost-management/v1/recommendations/openshift/settings/terms
DELETE /api/cost-management/v1/recommendations/openshift/settings/terms
```

Recommendation history and quality

Historical tracking of recommendation changes over time:

```
GET /api/cost-management/v1/recommendations/openshift/history
```

Recommendation quality metrics (accuracy of past predictions vs actual usage):

```
GET /api/cost-management/v1/recommendations/openshift/quality
```

Forecasts

Cost forecasts predict future spending based on historical trends.

OpenShift cost forecast

```
GET /api/cost-management/v1/forecasts/openshift/costs/
```

AWS cost forecast

```
GET /api/cost-management/v1/forecasts/aws/costs/
```

Azure cost forecast

```
GET /api/cost-management/v1/forecasts/azure/costs/
```

Google Cloud cost forecast

```
GET /api/cost-management/v1/forecasts/gcp/costs/
```

OCP-on-cloud cost forecasts

```
GET /api/cost-management/v1/forecasts/openshift/infrastructures/all/costs/
GET /api/cost-management/v1/forecasts/openshift/infrastructures/aws/costs/
GET /api/cost-management/v1/forecasts/openshift/infrastructures/azure/costs/
GET /api/cost-management/v1/forecasts/openshift/infrastructures/gcp/costs/
```

Resource Types

Resource type endpoints return the available values for group_by and filter parameters. Useful for building dropdowns in UIs or discovering what filter values exist.

OpenShift

```
GET /api/cost-management/v1/resource-types/openshift-clusters/
GET /api/cost-management/v1/resource-types/openshift-projects/
GET /api/cost-management/v1/resource-types/openshift-nodes/
GET /api/cost-management/v1/resource-types/openshift-virtual-machines/
GET /api/cost-management/v1/resource-types/openshift-gpu-vendors/
GET /api/cost-management/v1/resource-types/openshift-gpu-models/
```

AWS

```
GET /api/cost-management/v1/resource-types/aws-accounts/
GET /api/cost-management/v1/resource-types/aws-services/
GET /api/cost-management/v1/resource-types/aws-regions/
GET /api/cost-management/v1/resource-types/aws-organizational-units/
```

```
GET /api/cost-management/v1/resource-types/aws-categories/
```

Azure

```
GET /api/cost-management/v1/resource-types/azure-subscription-guids/
GET /api/cost-management/v1/resource-types/azure-services/
GET /api/cost-management/v1/resource-types/azure-regions/
```

Google Cloud

```
GET /api/cost-management/v1/resource-types/gcp-accounts/
GET /api/cost-management/v1/resource-types/gcp-projects/
GET /api/cost-management/v1/resource-types/gcp-services/
GET /api/cost-management/v1/resource-types/gcp-regions/
```

Cost models

```
GET /api/cost-management/v1/resource-types/cost-models/
```

Cost Distribution Fields

OpenShift cost report responses include distributed overhead costs when a cost model has platform/worker distribution enabled:

Field	Definition
cost_platform_distributed	Platform project overhead distributed proportionally to user projects (by CPU or memory weighting)
cost_worker_unallocated_distributed	Worker node unallocated capacity distributed proportionally to user projects
cost_total	Total cost including raw + usage + markup + platform_distributed + worker_unallocated_distributed

These fields appear in nested form in the API JSON response (e.g. `cost.platform_distributed`, `cost.worker_unallocated_distributed`, `cost.total`).

Settings

Platform projects (cost groups)

```
GET /api/cost-management/v1/settings/cost-groups/
```

Enabled tags

All enabled tags:

```
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true
```

Enabled tags filtered by source type:

```
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=OCP
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=AWS
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=Azure
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=GCP
```

Display currency

```
GET /api/cost-management/v1/account-settings/currency/
```

Cost models

List all cost models:

```
GET /api/cost-management/v1/cost-models/
```

Get rates on a single cost model by adding the cost model UUID:

```
GET /api/cost-management/v1/cost-models/{cost_model_uuid}/
```

Cost model relations are set at an OpenShift integration level (which includes one OpenShift cluster), or AWS/Azure/GCP integration level (which includes the parent and all child accounts in the same cloud integration). Everything covered by one integration shares the same cost model.

An integration can only be assigned to a single cost model, but multiple integrations (e. g. OpenShift clusters) can be assigned to the same cost model.

Matching data back to a cost model requires looking at the `source_uuid` value for each line item in the report endpoints, then searching for it in the cost-models endpoint.

Integrations

Formerly known as "sources", an integration is a cloud account or OpenShift cluster added to Cost Management. You can use the API to list sources added to your account or see last ingest completion times per source.

List all integrations:

```
GET /api/cost-management/v1/sources/
```

Get a specific integration:

```
GET /api/cost-management/v1/sources/{source_uuid}/
```

Check the stats on a particular integration:

```
GET /api/cost-management/v1/sources/{source_uuid}/stats/
```

Filtering and Pagination

Time scope filters

All cloud and OpenShift cost endpoints support the following filters:

`filter[resolution]`

`daily` or `monthly`. Sets report to daily or monthly breakdown.

`filter[time_scope_value]` and `filter[time_scope_units]`

Used together to set the scope value and unit for the returned data.

- `-10` or `-30` with `filter[time_scope_units]=day` for the last 10 or 30 days.
- `-1`, `-2`, or `-3` with `filter[time_scope_units]=month` for the last 1, 2, or 3 months.

You can also use explicit start and end dates:

```
?start_date=2022-09-01&end_date=2022-09-10
```

Exclude filter

Exclude specific values from results. Works with the same dimensions as `filter` and `group_by`:

```
GET /api/cost-
management/v1/reports/openshift/costs/?group_by[project]=*&exclude[project]=openshift-monitoring
GET /api/cost-management/v1/reports/aws/costs/?group_by[account]=*&exclude[account]=123456789012
```

Pagination

All list endpoints support `limit` and `offset` for pagination:

```
?limit=10&offset=0
```

The response `meta` object includes `count` (total items) and `links` includes `first`, `next`, `previous`, `last` URLs.

Get CSV or JSON Reports

Any of the report endpoints can return CSV format by setting the `Accept` header to `text/csv`.

Any of the report endpoints can return JSON format by setting the `Accept` header to `application/json`.

Current month aggregation by cluster:

```
curl --location -g \
  'https://console.redhat.com/api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-
1&filter[resolution]=monthly&group_by[cluster]=*' \
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

Current month aggregation by project:

```
curl --location -g \
  'https://console.redhat.com/api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-
1&filter[resolution]=monthly&group_by[project]=*' \
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

Current month grouped per tag:

```
curl --location -g \
  'https://console.redhat.com/api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-
1&filter[resolution]=monthly&group_by[tag:tag_key]=tag_value' \
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

Current month combined group_by with filter (projects grouped, filtered by cluster and node):

```
curl --location -g \
  'https://console.redhat.com/api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-
1&filter[resolution]=monthly&group_by[project]=*&filter[cluster]=my_cluster&filter[node]=my_node'
\
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

All of the above examples and additional parameter/filter options can be used with CSV export.

Basic Field Definitions

Field	Definition
sup_usage	Cost calculations from rates defined in the price list within a cost model when selecting Supplementary as the Calculation type
sup_raw	Not currently being used in the cost management backend
sup_markup	Markup set in the cost model, applied against sup_usage
sup_total	Total combined sup_cost: sup_usage + sup_markup
infra_usage	Cost calculations from rates defined in the price list within a cost model when selecting Infrastructure as the Calculation type
infra_raw	Cost coming directly from a cloud bill
infra_markup	Markup set in the cost model, applied against infra_raw and infra_usage
infra_total	Total combined infra_cost: infra_usage + infra_markup + infra_raw
cost_usage	Total cost of usage: sup_usage + infra_usage
cost_raw	Total cost of raw: sup_raw + infra_raw
cost_markup	Total cost of markup: sup_markup + infra_markup
cost_total	Total combined cost: cost_usage + cost_markup + cost_raw
cost_units	Cost currency unit for all cost metrics
capacity	Total hourly capacity of metric (Memory/core hours)
capacity_count	Max cores/Mem for a cluster
capacity_count_units	Unit for count (e.g. Core)
usage	Total hourly usage for a project/node/cluster and/or day/month depending on the query
usage_units	Unit for the usage/request/limit metrics (e.g. Core-hours)
request	Total hourly request for a project/node/cluster and/or day/month depending on the query
request_unused	Total hourly request that is not being utilised (difference between request and usage)
request_unused_percent	Percent value for unused cluster request based on (unused_request / capacity) * 100

Field	Definition
limit	Total hourly limit for a project/node/cluster and/or day/month depending on the query
gpu_request	Total GPU device requests for a project/node/cluster
gpu_limit	Total GPU device limits for a project/node/cluster
gpu_usage	Total GPU utilization for a project/node/cluster (in device-hours)